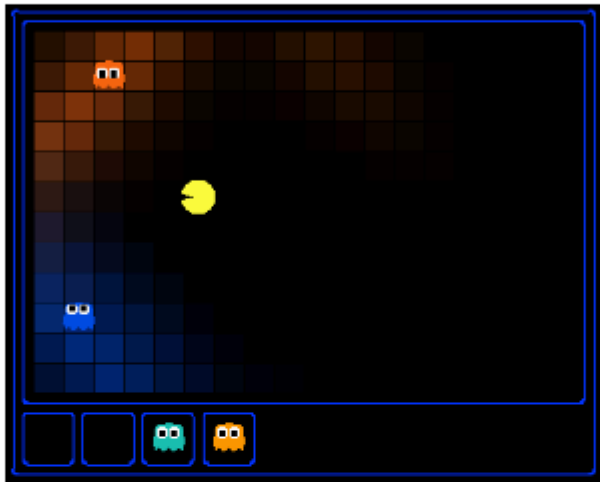




Projeto 4: Caça-Fantasmas

Prazo: quinta-feira, 15 de janeiro, 23h59 PT.



Eu posso te ouvir, fantasma.
Fugir não vai te salvar do meu
Filtro de partículas!

ÍNDICE

- Introdução Caça-Fantasmas
- e Bayes Nets
- Bayes Nets e FatoresPergunta 1 (2 pontos): Estrutura
- da Net BayesPergunta 2 (4 pontos): Juntar
- FatoresPergunta 3 (3 pontos): Eliminar (ainda não
- são fantasmas)Pergunta 4 (2 pontos): Eliminação de
- VariáveisPergunta 5a: Classe de Distribuição
- DiscretaPergunta 5b (2 pontos): Probabilidade de
- ObservaçãoPergunta 6 (2 pontos): Observação de
- Inferência ExataPergunta 7 (2 pontos): Inferência
- Exata com Tempo Decorrido

- Pergunta 8 (2 pontos): Teste Completo de Inferência Exata Pergunta
- 9 (2 pontos): Inicialização e Crenças de Inferência Aproximada
- Pergunta 10 (2 pontos): Observação de Inferência Aproximada
- Pergunta 11 (2 pontos): Inferência Aproximada com Tempo
- Decorrido Submissão

Introdução

Pacman passa a vida fugindo de fantasmas, mas as coisas nem sempre foram assim. Diz a lenda que, muitos anos atrás, o bisavô do Pacman, Grandpac, aprendeu a caçar fantasmas por esporte. No entanto, ele estava cego por seu poder e só conseguia rastrear fantasmas pelo barulho e barulho metálico. Neste projeto, você vai projetar agentes do Pacman que usam sensores para localizar e devorar fantasmas invisíveis. Você avançará de localizar fantasmas imóveis e isolados para caçar matilhas de múltiplos fantasmas em movimento com eficiência implacável. Este projeto inclui um autograder para você corrigir suas respostas na sua máquina. Isso pode ser berun em todas as perguntas com o comando:

```
Python autograder.py
```

Pode ser executado para uma pergunta específica, como q2, por:

```
Python autograder.py -q Q2
```

Ele pode ser executado para um teste específico por comandos do formulário:

```
python autograder.py -t test_cases/q1/1-0bsProb
```

O código deste projeto contém os seguintes arquivos, disponíveis como arquivo postal.

Arquivos que você vai editar:

bustersAgents.py	Agentes para jogar a variante Caça-Fantasmas do Pacman.
inference.py	Código para rastrear fantasmas ao longo do tempo usando seus sons.
factorOperations.py	Operações para calcular novas tabelas conjuntas ou de probabilidade marginalizadas.

Arquivos que você pode querer dar uma olhada:

bayesNet.py	As classes BayesNet e Factor.
Arquivos de suporte que você pode ignorar:	
busters.py	A entrada principal de Caça-Fantasmas (substituindo Pacman.py).
bustersGhostAgents.py	Novos agentes fantasmas para os Caça-Fantasmas.
distanceCalculator.py	Calcula distâncias de labirinto, armazena em cache resultados para evitar recomputação.
game.py	Funcionamento interno e aulas de ajuda para Pacman.
ghostAgents.py	Agentes para controlar fantasmas.
graphicsDisplay.py	Gráficos para Pacman.
graphicsUtils.py	Suporte para gráficos Pacman.
keyboardAgents.py	Interfaces de teclado para controlar o Pacman.
layout.py	Código para ler arquivos de layout e armazenar seu conteúdo.
util.py	Funções utilitárias.

Arquivos para editar e enviar: Você preencherá partes de `bustersAgents.py`, `inference.py`, e `factorOperations.py` durante a missão. Depois de concluir a tarefa, você enviará esses arquivos para o Gradescope (por exemplo, você pode enviar todos os arquivos .py na pasta). Por favor, não altere os outros arquivos desta distribuição.

Avaliação: Seu código será autoavaliado quanto à correção técnica. Por favor, não altere os nomes de nenhuma função ou classe fornecida dentro do código, ou você causará um caos no autograder. No entanto, a correção da sua implementação – não os julgamentos do autograder – será o juiz final da sua pontuação. Se necessário, revisaremos e corrigiremos as tarefas individualmente para garantir que você receba o devido crédito pelo seu trabalho.

Desonestidade Acadêmica: Vamos verificar seu código com outras submissões da aula para detectar redundância forlogica. Se você copiar o código de outra pessoa e enviar com pequenas alterações, saberemos. Esses detectores de trapanças são bem difíceis de enganar, então, por favor, não tente. Confiamos que todos vocês enviarão apenas seu próprio trabalho; Por favor, não nos decepcione. Se você o fizer, buscaremos as consequências mais fortes que possamos.

Buscando Ajuda: Você não está sozinho! Se você se sentir travado em algo, entre em contato com a equipe do curso para obter ajuda. Horário de atendimento, seção e fórum de discussão estão disponíveis para seu apoio; Por favor, use-os. Se você não puder comparecer ao nosso horário de atendimento, avise e agendaremos mais. Queremos esses

projetos para serem gratificantes e instrutivos, não frustrantes e desmoralizantes. Mas não sabemos quando ou como ajudar a menos que você pergunte. Discussão: Por favor, tome cuidado para não postar spoilers. Tópicos necessários para este projeto:

- Probabilidade: Vídeo 12, Nota 6.1-6.2
 - Representação de Redes Bayes: Vídeo 13, Nota
 - 6.3-6.4 Inferência de Redes Bayes: Vídeo 15,
 - Nota 6.6HMM: Vídeo 19, Nota 8.1-8.3 Filtragem
 - de partículas: Vídeo 20, Nota 8.4-8.5
-

Caça-Fantasmas e Redes Bayes

Na versão CS 188 de Caça-Fantasmas, o objetivo é caçar fantasmas assustados, mas invisíveis. Pacman, sempre engenhoso, está equipado com sonar (ouvidos) que fornece leituras ruidosas da distância de Manhattan para cada fantasma. O jogo termina quando Pacman comeu todos os fantasmas. Para começar, tente jogar um jogo você mesmo usando o teclado.

```
Python busters.py
```

Os blocos de cor indicam onde cada fantasma poderia estar, dado o ruído das leituras de distância fornecidos ao Pacman. As distâncias ruidosas na parte inferior da tela são sempre não negativas e sempre dentro de 7 da distância real. A probabilidade de uma leitura de distância diminui exponencialmente com sua diferença em relação à distância verdadeira. Sua tarefa principal neste projeto é implementar inferência para rastrear os fantasmas. No jogo baseado em teclado acima, uma forma rudimentar de inferência foi implementada por padrão: todos os quadrados onde um fantasma poderia estar são sombreados pela cor do fantasma. Naturalmente, queremos uma estimativa melhor da posição do fantasma. Felizmente, as Bayes Nets nos fornecem ferramentas poderosas para aproveitar ao máximo as informações que temos. Ao longo do restante deste projeto, você implementará algoritmos para realizar inferência exata e aproximada usando Bayes Nets. O projeto é desafiador, então recomendamos que você comece cedo e busque ajuda quando necessário. Enquanto observa e depura seu código com o autograder, será útil ter algum entendimento do que ele está fazendo. Existem 2 tipos de testes neste projeto, diferenciados por seus arquivos .test encontrados nos subdiretórios da pasta test_cases. Para testes da classe DoubleInferenceAgentTest, você verá visualizações das distribuições de inferência geradas pelo seu código, mas todas as ações do Pacman serão pré-selecionadas de acordo com as ações da implementação do staff. Isso é necessário para permitir a comparação das suas distribuições com as da equipe

distribuições. O segundo tipo de teste é o `GameScoreTest`, no qual seu `BustersAgent` realmente seleciona ações para Pacman e você assiste seu Pacman jogar e vencer partidas. Para este projeto, às vezes é possível que o autograder expire ao rodar os testes com gráficos. Para determinar com precisão se seu código é eficiente o suficiente, você deve rodar os testes com a flag `--no-graphics`. Se o autograder passar com essa bandeira, você receberá pontos completos, mesmo que o autograder tenha tempo com gráficos.

Redes Bayes e Fatores

Primeiro, dê uma olhada `bayesNet.py` para ver as classes com as quais você vai trabalhar – `BayesNet` e `Factor`. Você também pode rodar este arquivo para ver um exemplo de `BayesNet` e os `Factors` associados: `python bayesNet.py`. Você deve dar uma olhada na função `printStarterBayesNet` – há comentários úteis que podem facilitar muito sua vida no futuro. A `Bayes Net` criada nesta função é mostrada abaixo: (Chuva → Tráfego ← Jogo de bola) Um resumo da terminologia é apresentado abaixo:

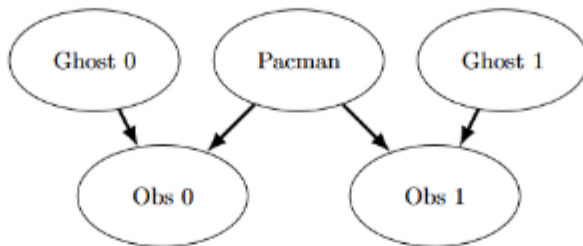
- **Bayes Net:** Esta é uma representação de um modelo probabilístico como um grafo direcionado acíclico e conjunto de tabelas condicionais de probabilidade, uma para cada variável, conforme mostrado na aula. A `Traffic Bayes Net` acima é um exemplo.
- **Fator:** Isso armazena uma tabela de probabilidades, embora a soma das entradas na tabela não seja necessariamente 1. Um fator $f_i(X_i | \text{pa}(X_i))$ para a variável X_i condicionada por suas pais $\text{pa}(X_i)$. Lembre-se de que variáveis minúsculas já foram atribuídas. Para cada possível combinação de valores das variáveis condicionadas, o fator armazena um único número. O fator f_i para a variável X_i condicionada por $\text{pa}(X_i)$ é denotado por $f_i(X_i | \text{pa}(X_i))$. Variáveis são incondicionadas. Tabela de Probabilidade Condicional (CPT): Este é um
- Suas entradas devem somar 1 para cada atribuição das variáveis condicionadas. 1. Existe exatamente uma variável incondicionada. A `Traffic Bayes Net` armazena os seguintes CPTs:
 - 1. $P(\text{Raining})$
 - 2. $P(\text{Ballgame} | \text{Raining})$
 - 3. $P(\text{Traffic} | \text{Ballgame}, \text{Raining})$

Pergunta 1 (2 pontos): Estrutura da Net Bayes

Implemente a função `constructBayesNet` em `inference.py`. Ela constrói uma Rede Bayes vazia com a estrutura descrita abaixo. Uma Rede Bayes é incompleta sem as probabilidades reais, mas os fatores são definidos e atribuídos separadamente pelo código da pauta; Você não precisa se preocupar com isso. Se você

Se você está curioso, você pode dar uma olhada em um exemplo de como funciona em `printStarterBayesNet` em `bayesNet.py`. Ler essa função também pode ser útil para fazer essa questão.

O mundo simplificado da caça a fantasmas é gerado de acordo com a seguinte rede Bayes:



Não se preocupe se isso parecer complicado! Vamos avançar passo a passo. Como descrito no código para `constructBayesNet`, construímos a estrutura vazia listando todas as variáveis, seus valores e as arestas entre elas. Esta figura mostra as variáveis e as arestas, mas e quanto aos domínios deles?

• Adicione variáveis e arestas com base no diagrama. Pacman e os dois fantasmas podem estar em qualquer lugar da grade (ignoramos paredes para isso). Adicione todas as tuplas de posição possível para esses. As observações aqui são não negativas, iguais às distâncias de Manhattan entre Pacman e ruído fantasma. Correção: Para testar e depurar $\pm$ seu código, execute

```
Python autograder.py -Q Q1
```

Pergunta 2 (4 pontos): Fatores de Junção

Implemente a função `joinFactors` em `factorOperations.py`. Ele recebe uma lista de Fatores `s` e retorna um novo Fator cujas entradas de probabilidade são o produto das linhas correspondentes da entrada Fatores `s`.

`joinFactors` podem ser usados como regra do produto, por exemplo, se tivermos um fator da forma $P(X / Y)$ e outro fator da forma $P(Y)$, então juntando esses fatores resultará $P(X, Y)$. Então `joinFactors` nos permite incorporar probabilidades para variáveis condicionadas (neste caso, E).

No entanto, você não deve assumir que `joinFactors` é chamado em tabelas de probabilidade – é possível chamar `joinFactors` em Fatores cujas linhas não somam 1. Gradação: Para testar e depurar seu código, execute

```
Python autograder.py -q Q2
```

Pode ser útil rodar testes específicos durante a depuração, para ver apenas um conjunto de fatores ser impresso. Por exemplo, para rodar apenas o primeiro teste, execute:

```
Python autograder.py -t test_cases/Q2/1-produto-regra
```

Dicas e Observações:

- Seus `joinFactors` devem devolver um novo `Factor`. Aqui
- estão alguns exemplos do que `joinFactors` pode fazer:
 - `joinFactors` ($P(X|Y)$, $P(Y)$) = $P(X, Y)$
 - `joinFactors` ($P(V, W | X, Y, Z)$, $P(X, Y | Z)$) = $P(V, W, X, Y | Z)$
 - `joinFactors` ($P(X|Y, Z)$, $P(Y)$) = $P(X, Y|Z)$
 - `joinFactors` ($P(V | W)$, $P(X | Y)$, $P(Z)$) = $P(V, X, Z | W, Y)$
- Para uma operação `joinFactors` geral, quais variáveis são incondicionadas no `Factor` retornado? Quais variáveis são condicionadas?
- O fator `s` armazena uma variável `DomíniosDicto`, que mapeia cada variável para uma lista de valores que pode assumir (seu domínio). Um `Fator` obtém seu `variableDomainsDict` da `BayesNet` da qual foi instanciado. Como resultado, ele contém todas as variáveis da `BayesNet`, não apenas as variáveis incondicionadas e condicionadas usadas no `Fator`. Para esse problema, pode-se assumir que todos os fatores de entrada `s` vieram da mesma `BayesNet`, e assim seus `variableDomainsDicts` são todos iguais.

Pergunta 3 (3 pontos): Eliminar (ainda não fantasmas)

Implemente a função de eliminar em `factorOperations.py`. Ele precisa de um `Fator` e uma variável para eliminar e retorna um novo `Fator` que não contém essa variável. Isso corresponde à soma de todas as entradas no `Fator` que diferem apenas no valor da variável que está sendo eliminada. Correção: Para testar e depurar seu código, execute

```
Python autograder.py -q Q3
```

Pode ser útil rodar testes específicos durante a depuração, para ver apenas um conjunto de fatores ser impresso. Por exemplo, para rodar apenas o primeiro teste, execute:

```
Python autograder.py -t test_cases/q3/1-simple-eliminar
```

Dicas e Observações:

- Sua eliminação deve devolver um novo Fator .
- `eliminar` pode ser usado para marginalizar variáveis de tabelas de probabilidade. Por exemplo:
 - `eliminar(P(X, Y | Z), Y) = P(X | Z)`
 - `eliminar(P(X, Y | Z), X) = P(Y | Z)`
- Para uma operação geral de eliminação, quais variáveis são incondicionadas no Fator retornado? Quais variáveis são condicionadas? Lembre-se de que o Fator `s` armazena a variável `DomíniosDict` da `BayesNet` original , e não apenas as variáveis incondicionadas e condicionadas que eles utilizam. Como resultado, o Fator retornado deve ter a mesma variável `DomíniosDict` que o Fator de entrada .

Pergunta 4 (2 pontos): Eliminação Variável

Implemente a função `inferenceByVariableElimination` em `inference.py` . Ele responde a uma pergunta probabilística, que é representada usando um `BayesNet`, uma lista de variáveis de consulta e as evidências. Correção: Para testar e depurar seu código, execute

```
Python autograder.py -q Q4
```

Pode ser útil rodar testes específicos durante a depuração, para ver apenas um conjunto de fatores ser impresso. Por exemplo, para rodar apenas o primeiro teste, execute:

```
Python autograder.py -t test_cases/q4/1-desconectado-eliminar
```

Dicas e Observações:

- O algoritmo deve iterar sobre variáveis ocultas em ordem de eliminação, realizando `joiningover` e eliminando essa variável, até que restem apenas as variáveis de consulta e evidência. A soma das probabilidades no seu fator de saída deve somar 1 (de modo que seja uma probabilidade verdadeira condicional, condicionada à evidência). Veja a função `InferenceByEnumeration` em `inference.py` para um exemplo de como usar as funções desejadas. (Lembrete: Inferência por enumeração primeiro junta todas as variáveis e depois elimina todas as variáveis ocultas. Em contraste, a eliminação de variáveis intercala `join` e eliminação iterando sobre todas as variáveis ocultas e realiza um `join and eliminate` em uma única variável oculta antes de passar para a próxima variável oculta.)

- Você precisará cuidar do caso especial em que um fator que você juntou tem apenas uma variável não condicionada (o docstring especifica o que fazer com mais detalhes).

Pergunta 5a: Classe DiscreteDistribution

Infelizmente, ter passos de tempo vai aumentar nosso gráfico demais para que a eliminação de variáveis seja viável. Em vez disso, usaremos o Algoritmo Forward para HMMs para inferência exata, e ParticleFiltering para inferência ainda mais rápida, porém aproximada. Para o restante do projeto, usaremos a classe DiscreteDistribution definida no modelo inference.py distribuições de crença e distribuições de peso. Essa classe é uma extensão da classe de dicionário Python embutida, onde as chaves são os diferentes elementos discretos da nossa distribuição, e os valores correspondentes são proporcionais à crença ou peso que a distribuição atribui a esse elemento. Esta pergunta pede que você preencha as partes que faltam nesta aula, o que será crucial para questões futuras (mesmo que essa pergunta em si não valha pontos). Primeiro, preencha o método de normalização, que normaliza os valores na distribuição para somar um, mas mantém as proporções dos valores iguais. Use o método total para encontrar a soma dos valores na distribuição. Para uma distribuição vazia ou uma distribuição onde todos os valores são zero, não faça nada. Note que esse método modifica a distribuição diretamente, em vez de retornar uma nova distribuição. Segundo, preencha o método da amostra, que extrai uma amostra da distribuição, onde a probabilidade de uma chave ser amostrada é proporcional ao seu valor correspondente. Assuma que a distribuição não é vazia, e nem todos os valores são zero. Note que a distribuição não precisa necessariamente ser normalizada antes de chamar esse método. Você pode encontrar o Python embutido função `random.random()` útil para esta pergunta.

Pergunta 5b (2 pontos): Probabilidade de observação

Nesta pergunta, você implementará o método `getObservationProb` na classe base `InferenceModule` em `inference.py`. Esse método absorve uma observação (que é uma leitura ruidosa da distância ao fantasma), a posição do Pacman, a posição do fantasma e a posição da prisão do fantasma, e retorna a probabilidade da leitura da distância barulhenta dada a posição do Pacman e da posição do fantasma ($P(\text{ruidosoDistância} \mid \text{posição pacman}, \text{posição ghost})$).

O sensor de distância possui uma distribuição de probabilidade sobre leituras de distância dada a distância real do Pacman até o fantasma. Essa distribuição é modelada pela função `busters.getObservationProbability(ruidosoDistância, trueDistância)`, que retorna $P(\text{ruidosoDistância} \mid \text{trueDistance})$ e é providenciado para você. Você deve usar essa função para ajudar a resolver o problema

e usar a função `manhattanDistance` fornecida para encontrar a distância entre a localização do Pacman e a localização do fantasma. No entanto, há um caso especial de prisão que também precisamos lidar. Especificamente, quando capturamos um fantasma e o enviamos para o local da prisão, nosso sensor de distância retorna deterministicamente Nenhum e nada mais (observação = Nenhum se e somente se o fantasma estiver na prisão). Uma consequência disso é que, se a posição do fantasma é a posição de jail, então a observação é Nenhum com probabilidade 1, e todo o resto com probabilidade 0. Certifique-se de lidar com esse caso especial na sua implementação; efetivamente temos um conjunto diferente de regras para quando Ghost está preso, assim como para quando a observação é Nenhuma. Para testar seu código e rodar o autograder para esta pergunta:

```
Python autograder.py -q Q5
```

Pergunta 6 (2 pontos): Observação de Inferência Exata

Nesta pergunta, você implementará o método `observeUpdate` na classe `ExactInference` de `inference.py` atualizar corretamente a distribuição de crenças do agente sobre as posições dos fantasmas, dado uma observação dos sensores do Pacman. Você está implementando a atualização online de crenças para observar novas evidências. O método `observeUpdate` deve, para esse problema, atualizar a crença em todas as posições do mapa após receber uma leitura do sensor. Você deve iterar suas atualizações sobre `thevariable self.allPositions`, que inclui todas as posições jurídicas mais a posição especial na prisão. Crenças representam a probabilidade de o fantasma estar em um determinado local, e são armazenadas como um Objeto `DiscreteDistribution` em um campo chamado `self.beliefs`, que você deve atualizar.

Antes de digitar qualquer código, escreva a equação do problema de inferência que você está tentando resolver. Você deve usar a função `self.getObservationProb` que você escreveu na última pergunta, que retorna a probabilidade de observação dada a posição do Pacman, uma potencial ghostposition e a posição na prisão. Você pode obter a posição do Pacman usando `gameState.getPacmanPosition()`, e a posição na prisão usando `self.getJailPosition()`.

Na exibição do Pacman, crenças posteriores altas são representadas por cores vivas, enquanto crenças baixas são representadas por cores fracas. Você deve começar com uma grande nuvem de crença que diminui com o tempo à medida que mais evidências se acumulam. Ao observar os casos de teste, certifique-se de entender como os quadrados convergem para sua coloração final. Nota: seus agentes do Busters têm um módulo de inferência separado para cada fantasma que estão rastreando. Por isso, se você imprimir uma observação dentro da função `observeUpdate`, verá apenas um número, mesmo que possa haver vários fantasmas no quadro. Para rodar o autograder para esta pergunta e visualizar o resultado:

Se quiser rodar este teste (ou qualquer um dos outros testes) sem gráficos, pode adicionar a seguinte bandeira:

```
Python autograder.py -q Q6 --Não-Gráficos
```

Pergunta 7 (2 pontos): Inferência Exata com Tempo Decorrido

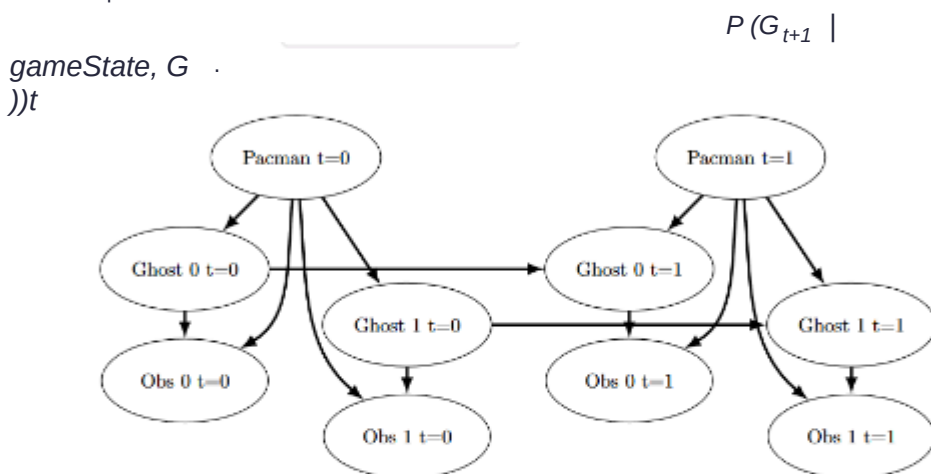
Na pergunta anterior, você implementou atualizações de crenças para o Pacman com base nas observações dele. Felizmente, as observações do Pacman não são sua única fonte de conhecimento sobre onde um fantasma pode estar. Pacman também tem conhecimento sobre as formas como um fantasma pode se mover; ou seja, que o fantasma não pode atravessar uma parede ou mais de um espaço em um único intervalo de tempo. Para entender por que isso é útil para o Pacman, considere o seguinte cenário em que há o Pacman e um Fantasma. Pacman recebe muitas observações que indicam que o fantasma é muito orçado, mas uma que indica que o fantasma está muito longe. A leitura que indica que o fantasma está muito distante provavelmente é resultado de um sensor com bugs. O conhecimento prévio do Pacman sobre como o fantasma pode mover diminuirá o impacto dessa leitura, já que Pacman sabe que o fantasma não poderia se mover tão longe em apenas um movimento. Nesta pergunta, você implementará o método `elapseTime` em `ExactInference`. O passo de tempo de `elapse`, para esse problema, deve atualizar a crença em cada posição do mapa após um passo de tempo decorrido. Seu agente tem acesso à distribuição de ações do fantasma através

`self.getPositionDistribution`. Para obter a distribuição sobre novas posições para o fantasma, dada sua posição anterior, use esta linha de código:

```
newPosDist = self.getPositionDistribution(gameState, oldPos)
```

Onde `oldPos` se refere à posição fantasma anterior. `newPosDist` é um objeto de Distribuição Discreta, onde para cada posição `p` em `self.allPositions`, `newPosDist[p]` é a probabilidade de que o fantasma esteja em posição `p` no tempo $t + 1$, dado que o fantasma está na posição `oldPos` no tempo t . Note que essa chamada pode ser relativamente cara, então, se seu código está expirando, uma coisa a considerar é se você pode ou não reduzir o número de chamadas para `self.getPositionDistribution`. Antes de digitar qualquer código, escreva a equação do problema de inferência que você está tentando resolver. Para testar sua implementação de previsão separadamente da sua implementação de atualização na pergunta anterior, esta pergunta não fará uso da sua implementação de atualização.

Como Pacman não está observando as ações do fantasma, essas ações não impactarão suas crenças. Com o tempo, as crenças do Pacman passarão a refletir lugares no tabuleiro onde ele acredita que os fantasmas são mais prováveis, dado a geometria do tabuleiro e os possíveis movimentos legais dos fantasmas, que o Pacman já conhece. Para os testes desta pergunta, às vezes usamos um fantasma com movimentos aleatórios e outras vezes usamos o GoSouthGhost. Esse fantasma tende a se mover para o sul, então, com o tempo, e sem nenhuma observação, a distribuição de crenças do Pacman deve começar a se concentrar na parte inferior do tabuleiro. Para ver qual ghost é usado em cada caso de teste, você pode olhar nos arquivos .test. O diagrama abaixo mostra o que a Rede Bayes/modelo de Markov Oculto mostra o que está acontecendo. Ainda assim, você deve confiar na descrição acima para a implementação porque algumas partes são implementadas para você (ou seja, getPositionDistribution é abstraído para ser



Para rodar o autograder para esta pergunta e visualizar o resultado:

```
Python autograder.py -q Q7
```

Se quiser rodar este teste (ou qualquer um dos outros testes) sem gráficos, pode adicionar a seguinte bandeira:

```
Python autograder.py -q Q7 --Sem Gráficos
```

Enquanto você observa o resultado do autograder, lembre-se que quadrados mais claros indicam que o pacman acredita que um fantasma tem mais chance de ocupar aquele local, e quadrados mais escuros indicam que um fantasma tem menos chance de ocupar aquele local. Para qual dos casos de teste você percebe diferenças na sombra dos quadrados? Você pode explicar por que alguns quadrados ficam mais claros e outros mais escuros?

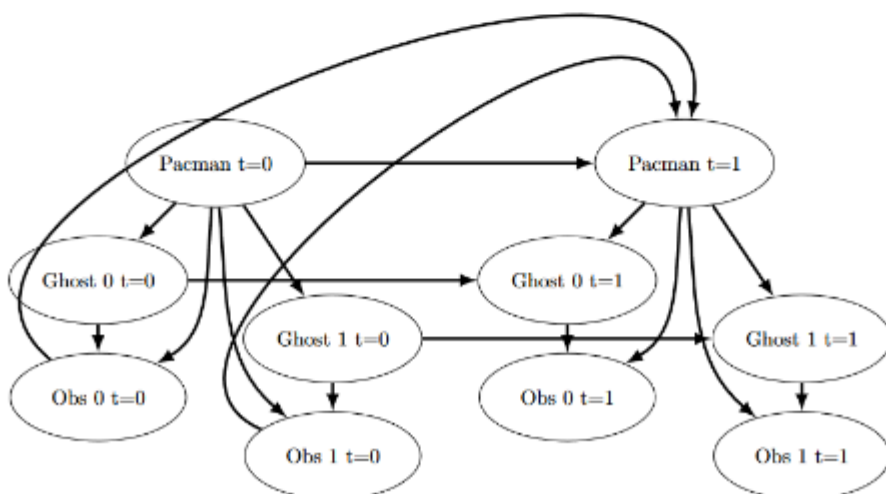
Pergunta 8 (2 pontos): Teste Completo de Inferência Exata

Agora que Pacman sabe usar tanto seu conhecimento prévio quanto suas observações ao descobrir onde está um fantasma, ele está pronto para caçar fantasmas sozinho. Vamos usar sua Implementações observeUpdate e elapseTime juntas para manter uma distribuição de crenças atualizada, e seu agente ganancioso simples escolherá uma ação baseada nas distribuições mais recentes em cada passo de tempo. Na estratégia simples e gananciosa, Pacman assume que cada fantasma está em sua posição mais provável de acordo com suas crenças, então se dirige ao fantasma mais próximo. Até esse ponto, Pacman se moveu selecionando aleatoriamente uma ação válida.

Implemente o método chooseAction no GreedyBustersAgent em bustersAgents.py . Seu agente deve primeiro encontrar a posição mais provável de cada fantasma não capturado restante, depois escolher uma ação que minimize a distância do labirinto até o fantasma mais próximo. Para encontrar a distância do labirinto entre quaisquer duas posições pos1 e pos2 , use `self.distancer.getDistance(pos1, pos2)`. Para encontrar a posição sucessora de uma posição após uma ação:

```
sucessorPosição = Ações.Sucessor(posição, ação)
```

Você recebe livingGhostPositionDistributions, uma lista de objetos DiscreteDistribution que representa as distribuições de crença de posição para cada um dos fantasmas que ainda não foram capturados. Se implementado corretamente, seu agente deve vencer o jogo no teste de pontuação de jogo de 8/3 com uma pontuação maior que 700 pelo menos 8 em cada 10 vezes. Nota: o autoavaliador também verifica diretamente a correção da sua inferência, mas o resultado dos jogos é um teste razoável de sanidade. Podemos representar como nosso agente ganancioso trabalha com a seguinte modificação ao diagrama anterior:



Para rodar o autograder para esta pergunta e visualizar o resultado:

```
Python autograder.py -q Q8
```

Se quiser rodar este teste (ou qualquer um dos outros testes) sem gráficos, pode adicionar a seguinte bandeira:

```
Python autograder.py -q Q8 --Sem Gráficos
```

Pergunta 9 (2 pontos): Inicialização e Crenças Aproximadas de Inferência

A inferência aproximada está muito em alta entre caçadores de fantasmas nesta temporada. Nas próximas perguntas, você implementará um algoritmo de filtragem de partículas para rastrear um único fantasma. Primeiro, implemente as funções `initializeUniformly` e `getBeliefDistribution` na classe `ParticleFilter` em `inference.py`. Uma partícula (amostra) é uma posição fantasma nesse problema de inferência. Note que, para inicialização, as partículas devem ser distribuídas uniformemente (não aleatoriamente) entre posições legais para garantir um prior uniforme. Recomendamos pensar em como o operador `mod` é útil para `initializeUniformly`. Note que a variável em que você armazena suas partículas deve ser uma lista. Uma lista é simplesmente uma coleção de variáveis ponderadas com `fum` (posições neste caso). Armazenar suas partículas como qualquer outro tipo de dado, como um dicionário, está incorreto e gera erros. O método `getBeliefDistribution` então pega a lista de partículas e a converte em um objeto `DiscreteDistribution`. Para testar seu código e rodar o autograder para esta pergunta:

```
Python autograder.py -q Q9
```

Pergunta 10 (2 pontos): Observação por Inferência Aproximada

Em seguida, implementaremos o método `observeUpdate` na classe `ParticleFilter` em `inference.py`. Esse método constrói uma distribuição de pesos sobre as próprias partículas onde o peso de uma partícula é a probabilidade da observação dada a posição do Pacman e a localização dessa partícula. Depois, `wereamostre` dessa distribuição ponderada para construir nossa nova lista de partículas. Você deve novamente usar a função `self.getObservationProb` para encontrar a probabilidade de uma observação dada a posição do Pacman, uma possível posição fantasma e a posição na prisão. O método de amostragem da classe `DiscreteDistribution` também será útil. Para lembrar, você pode obter

A posição do Pacman usando `gameState.getPacmanPosition()` , e a posição na prisão usando `self.getJailPosition()` .

Há um caso especial que uma implementação correta deve lidar. Quando todas as partículas recebem peso zero, a lista de partículas deve ser reinicializada chamando o `initializeUniformly` . O método `total` da `DiscreteDistribution` pode ser útil. Para rodar o autograder para esta pergunta e visualizar o resultado:

```
Python autograder.py -q Q10
```

Se quiser rodar este teste (ou qualquer um dos outros testes) sem gráficos, pode adicionar a seguinte bandeira:

```
Python autograder.py -q Q10 --Sem-Gráficos
```

Pergunta 11 (2 pontos): Inferência Aproximada com Tempo Passado

Implemente a função `elapseTime` na classe `ParticleFilter` em `inference.py` . Essa função deve construir uma nova lista de partículas que corresponda a cada partícula existente em `self.partículas` avançando um passo de tempo, e então atribuir essa nova lista de volta a `self.partículas` . Quando terminado, você deve conseguir rastrear fantasmas quase tão eficazmente quanto com inferências exatadas. Note que, nesta pergunta, testaremos tanto a função `elapseTime` isoladamente, quanto a implementação completa do filtro de partículas que combina `elapseTime` e observaremos . Como no método `elapseTime` da classe `ExactInference`, você deve usar:

```
newPosDist = self.getPositionDistribution(gameState, oldPos)
```

Essa linha de código obtém a distribuição sobre novas posições para o fantasma, dada sua posição anterior (`oldPos`). O método de amostra da classe `DiscreteDistribution` também será útil. Para rodar o autograder para esta pergunta e visualizar o resultado:

```
Python autograder.py -q Q11
```

Se quiser rodar este teste (ou qualquer um dos outros testes) sem gráficos, pode adicionar a seguinte bandeira:

```
Python autograder.py -q Q11 --Sem Gráficos
```

Note que, mesmo sem gráficos, esse teste pode levar vários minutos para ser executado.

Submissão

Para enviar seu projeto, envie os arquivos Python que você editou. Por exemplo, o upload do useGradescope em todos os arquivos .py na pasta do projeto.
